HOMEWORK 3...? 🕵️‍♂️

DUE: *October 21, 2025 @ 11:59 PM*

24-HR LATE DUE DATE W/ A 15% PENALTY: *October 22, 2025 @ 11:59 PM*

DO NOT REMOVE ANY PART OF ANY OF THE QUESTIONS OR YOU LOSE CREDIT

No Hardcoding Any Part of Any Question 😊 🪄 🧌

REMEMBER TO SHOW ALL CODE OUTPUT (NO CREDIT OTHERWISE)

Reminder: Use tools or websites such as https://www.convert.ploomber.io/ to create a clean PDF from jupyter, check piazza for more details

Reminder: Please make sure your code runs before submitting your work. Code sections that do not run will receive 0 credits, no partials will be given. This is VERY important in real project development.

DISCLAIMER: This is a fake story, not a real incident! We have created this story just for the data exploration purpose.

Campus Police Department

Incident Report

Incident Date: September 27, 2025

Location of Incident: 2nd Floor, Iribe Building

Items Left at the Scene: Red Ticket for Zichao's Magic Show

Case Number: 2025-IRB-0027

Victim Information:

Name: Dr. Fardina Alam

Affiliation: University of Maryland, College Park

Position: Professor

Contact Information: fardina@umd.edu

Incident Description:

On the evening of September 27, 2025, an unknown individual(s) unlawfully entered Dr. Fardina's room located on the 2nd floor of Iribe and stole a set of Markers. The stolen items were a set of limited edition markers that Dr. Fardina had spent years collecting. At the scene of the crime, a red entry ticket for "Zichao's Magic Show" was found on the floor near Dr. Fardina's desk.

In [98]:
```
# We import all essential libraries for this HW.
# If you want to add additional libraries, you can add more here!
import pandas as pd
from datetime import datetime
import string
import math
import re
```

YOUR TASK 🔎:

In this assignment, you are tasked with solving the mystery of Dr. Fardina's stolen markers and finding the culprit.

- To accomplish this, you will be provided with **4 data sets** containing key information related to the incident. The list of the data sets name given below:

- Profiles
- Witnesses
- Zichao's magic show
- Interrogation Statements

- Your goal is to analyze these data sets to uncover the clues and evidences that will lead you to the thief.

Be careful in following instructions as it may lead to you losing points :(

Take a moment to look at all the data sets below:

Profiles:

This dataset contains information regarding all the potential people that could have committed the crime. General information regarding each person are as follows:

- `ID` : A **unique** identification for each person
- `Name` : The name of the person
- `Birthday` : The birthday of the person
- `Location_during_crime` : The location of the person during the crime
- `Eye_Color` : The eye color of the person
- `Skin_Color` : The skin color of the person
- `Hair_Color` : The hair color of the person
- `Height` : The height of the person in cm

Witnesses:

This dataset contains a statement from everyone in profiles, regarding suspicious things they saw during the time of the crime. Information that comes in this dataset are as follows:

- `ID` : A **Unique** identification for each person
- `Statement` : A statement from the person regarding a suspicious activity during the time of the crime

Zichao's Magic Show:

This dataset contains everyone that purchased a ticket for the show. **NOTE** that not everyone in profiles has purchased a ticket for this show. The Information contained in this dataset are as follows:

- `ID` : A **Unique** indentification for each person
- `Ticket_Number` : A **Unique** number that serves as an identification for the ticket bought

Interrogation Statements:

This datasets contains a statement from each person after they have been interrogated. **NOTE** assume that each statement is ***100% true*** and that ***there are people in this dataset who aren't in the "Profiles" dataset***. Information contained in this dataset are as follows:

- `Name` : The name of the person being interrogated
- `Statement` : A statement the person gives after being interrogated

---

Q1) Lets begin by observing the profiles dataset (10 PTS Total)

TASK 1.1 (1 Point): In the code box below, load in `Profiles` dataset into a variable called `profiles_df` and display it

Outputs to be Graded:

- Variable(s): `profiles_df`
- Cell outputs

In [99]:
```
profiles_df = pd.read_csv("Profiles.csv")
display(profiles_df)
```

	ID	Name	Birthday	Location_during_crime	Eye_Color	Skin_Color	Hair_Color	Height
0	P299	Marco Rossi	1969/07/10	fourth floor	red	indigo	orange	191
1	P086	Aarav Patel	1979/03/10	second floor	indigo	indigo	green	198
2	P022	Li Wei	1990/10/08	fourth floor	blue	red	violet	154
3	P003	Haruto Tanaka	1996/27/10	lobby	green	blue	violet	158
4	P036	Kwame Mensah	1991/26/02	fourth floor	blue	yellow	orange	184
...	...	...	...	...	...	...	...	...
343	P126	Andrew Choe	1988/03/01	NaN	violet	indigo	red	176
344	P203	Krish Thakker	1965/26/09	third floor	yellow	indigo	orange	193
345	P273	Jeonghan Yoon	2003/12/07	fourth floor	blue	indigo	red	193
346	P290	Ananya Malipeddi	10/03/1994	third floor	NaN	yellow	indigo	161
347	P233	Ifra Syed	06/27/1982	second floor	red	yellow	orange	169

348 rows × 8 columns

TASK 1.2 (1 Point): Next, you need to display the number of rows and columns in the `profiles_df` dataframe.

Outputs to be Graded:

- Cell outputs

In [100…
```
display(profiles_df.shape)
```

```
(348, 8)
```

TASK 1.3 (1 Point): Next, you need to display the name of the columns in the `profiles_df` dataframe.

Outputs to be Graded:

- Cell outputs

In [101…
```
for i in profiles_df.columns:
    display(i)
```

```
'ID'
'Name'
'Birthday'
'Location_during_crime'
'Eye_Color'
'Skin_Color'
'Hair_Color'
'Height'
```

TASK 1.4 (1 Point): Now, you need to display the types of the columns in the `profiles_df` dataframe.

Outputs to be Graded

- Cell outputs

In [102…
```
display(profiles_df.dtypes)
```

```
ID                     object
Name                   object
Birthday               object
Location_during_crime  object
Eye_Color              object
Skin_Color             object
Hair_Color             object
Height                  int64
dtype: object
```

TASK 1.5 (2 Points):

- Utilizing the `count()` function on this dataset and set it to the variable `info` and then display it.
- Write any **potential problem** that you see with the dataset when observing the output of the count function:

Outputs to be Graded

- Variable(s): `info`

- Cell outputs
- Written statement(s) describing potential problem(s)

In [103…
```
info = profiles_df.count()
display(info)
```

```
ID                       348
Name                     348
Birthday                 348
Location_during_crime    280
Eye_Color                280
Skin_Color               348
Hair_Color               348
Height                   348
dtype: int64
```

Write your observations here: Eye color and Location_during_crime don't have the same counts as every other column- there is missing data

TASK 1.6 (4 Points):

Look back at the "Profiles" dataset. Using other pandas functions than `count()`, **can you spot another potential problem with the dataset?**

Outputs to be Graded

- Written statement(s) describing your observations

In [104…
```
display(profiles_df["Skin_Color"].value_counts())
display(profiles_df["Birthday"].value_counts())
display(profiles_df["ID"].value_counts())
```

```
Skin_Color
indigo    56
blue      55
yellow    54
orange    51
violet    49
green     42
red       41
Name: count, dtype: int64
Birthday
1979/20/07    3
1989/08/09    3
2000/30/10    2
1969/07/10    2
11/07/2000    2
             ..
1969/23/07    1
1980/08/09    1
07/02/1994    1
1991/07/12    1
01/04/1987    1
Name: count, Length: 297, dtype: int64
ID
P141    2
P139    2
P156    2
P158    2
P296    2
       ..
P048    1
P279    1
P098    1
P147    1
P086    1
Name: count, Length: 300, dtype: int64
```

Write your observations here: Birthday column has inconsistent format and there are duplicates of IDs

Q2) Cleaning the data set (25 PTS Total)

TASK 2.1 (10 Points): Lets begin by fixing the Birthday column. You may notice that the dates are in inconsistent format!

- Pandas internally represent dates in 'datetime64' format which is a form of ISO 8601. Basically, dates are represented as yyyy-mm-dd.
- Fix all the dates in the Birthday Column of `profiles_df` dataframe (Do not use other variable name).

- Display the `profiles_df` dataframe after you are done.
- Note that the only **2 formats** the dates are in is either **yyyy/dd/mm** or **mm/dd/yyyy**;
- You want to represent them as **yyyy-mm-dd**
- **EXAMPLE**: In the "Profiles" dataset, dates like *2000/10/12* and *10/12/2000* follow different formats—one as *December 10th, 2000*, and the other as *October 12th, 2000*. We aim to standardize them to *2000-12-10* and *2000-10-12*, respectively.

HINT: some datetime functions might make things easier.

- https://www.geeksforgeeks.org/python-pandas-to_datetime/
- https://www.programiz.com/python-programming/datetime/strftime

Outputs to be Graded

- Variable(s): `profiles_df`
- Cell outputs

In [105…
```
date1 = pd.to_datetime(profiles_df["Birthday"], errors="coerce", format="%Y/%d/%m")
date2 = pd.to_datetime(profiles_df["Birthday"], errors="coerce", format="%m/%d/%Y")

profiles_df["Birthday"] = date1.fillna(date2)
display(profiles_df)
```

	ID	Name	Birthday	Location_during_crime	Eye_Color	Skin_Color	Hair_Color	Height
0	P299	Marco Rossi	1969-10-07	fourth floor	red	indigo	orange	191
1	P086	Aarav Patel	1979-10-03	second floor	indigo	indigo	green	198
2	P022	Li Wei	1990-08-10	fourth floor	blue	red	violet	154
3	P003	Haruto Tanaka	1996-10-27	lobby	green	blue	violet	158
4	P036	Kwame Mensah	1991-02-26	fourth floor	blue	yellow	orange	184
...	...	...	...	...	...	...	...	...
343	P126	Andrew Choe	1988-01-03	NaN	violet	indigo	red	176
344	P203	Krish Thakker	1965-09-26	third floor	yellow	indigo	orange	193
345	P273	Jeonghan Yoon	2003-07-12	fourth floor	blue	indigo	red	193
346	P290	Ananya Malipeddi	1994-10-03	third floor	NaN	yellow	indigo	161
347	P233	Ifra Syed	1982-06-27	second floor	red	yellow	orange	169

348 rows × 8 columns

TASK 2.2 (5 Points): Now that we have cleaned up the Birthday column, lets find the age of each person. Create a new column called `Age` that displays the age of the person **on the day of the crime.** Once you are done, display the `profiles_df` dataframe

Outputs to be Graded

- Variable(s): `profiles_df`
- Cell outputs

In [106…
```
profiles_df["Age"] = (datetime(2025, 9, 25, 0, 0, 0, 0) - profiles_df["Birthday"]).dt.components.days//365
display(profiles_df)
```

	ID	Name	Birthday	Location_during_crime	Eye_Color	Skin_Color	Hair_Color	Height	Age
0	P299	Marco Rossi	1969-10-07	fourth floor	red	indigo	orange	191	56
1	P086	Aarav Patel	1979-10-03	second floor	indigo	indigo	green	198	46
2	P022	Li Wei	1990-08-10	fourth floor	blue	red	violet	154	35
3	P003	Haruto Tanaka	1996-10-27	lobby	green	blue	violet	158	28
4	P036	Kwame Mensah	1991-02-26	fourth floor	blue	yellow	orange	184	34
...	...	...	...	...	...	...	...	...	...
343	P126	Andrew Choe	1988-01-03	NaN	violet	indigo	red	176	37
344	P203	Krish Thakker	1965-09-26	third floor	yellow	indigo	orange	193	60
345	P273	Jeonghan Yoon	2003-07-12	fourth floor	blue	indigo	red	193	22
346	P290	Ananya Malipeddi	1994-10-03	third floor	NaN	yellow	indigo	161	31
347	P233	Ifra Syed	1982-06-27	second floor	red	yellow	orange	169	43

348 rows × 9 columns

TASK 2.3 (10 Points): Now that we have created the `Age`; lets clean the `Eye_Color` column.

Please read the directions carefully:

- In order to fill in the missing eye colors, we want to find the **most common eye color** among those who have the **same hair and skin color**.
- For instance, if we want to find the eye color of someone who has yellow hair and violet skin:
- we would first find everyone who has the same hair and skin color, in this case it would be yellow hair and violet skin
- and then we find the most common eye color among them.
- we then replace the missing eye color with the eye color that is the most common among the people with the same hair and skin color.
- Watch out for redundant data!
- ***NOTE: In the case that you find multiple modes, choose the color that appears first in alphabetical order***

When you finish filling in the missing eye colors, please **display the** `profiles_df`

Outputs to be Graded

- Variable(s): `profiles_df`
- Cell outputs

In [151…

```
# display(profiles_df[profiles_df["Hair_Color"] == profiles_df["Skin_Color"]].groupby(["Hair_Color", "Skin_Color"])["Eye_Color
eye_colors = {}

def helper(x):
    common = x.mode()[0]
    # eye_colors[x.name[0]] = common
    # display(eye_colors)
    x.fillna(common, inplace=True)
    return x

# def helper2(x):
    # if
# display(profiles_df[profiles_df["Hair_Color"] == profiles_df["Skin_Color"]].groupby(["Hair_Color", "Skin_Color"])["Eye_Color

profiles_df[profiles_df["Hair_Color"] == profiles_df["Skin_Color"]].groupby(["Hair_Color", "Skin_Color"])["Eye_Color"].apply(l
# profiles_df["Eye_Color"] = profiles_df.groupby(["Hair_Color", "Skin_Color"])["Eye_Color"].apply(lambda x: helper(x))

# # display(eye_colors)

# profiles_df[profiles_df["Hair_Color"] == profiles_df["Skin_Color"]].apply(lambda x: helper2(x), axis=1)
# # display(profiles_df)
```

Out[151…

Hair_Color	Skin_Color		
blue	blue	35	orange
		83	violet
		118	green
		149	green
		178	blue
		208	green
		215	green
		272	green
		274	indigo
		322	green
green	green	28	orange
		132	red
		133	green
		197	green
		202	violet
		205	red
indigo	indigo	42	orange
		52	orange
		101	yellow
		159	violet
		161	yellow
		221	blue
		225	orange
		318	orange
orange	orange	114	violet
		117	green
		154	green
		308	green
red	red	9	blue
		46	red
		94	indigo
		126	blue
		152	yellow
		195	orange
		218	red
		243	red
		245	red
		248	orange
		262	green
		270	violet
		277	green
		332	red
violet	violet	7	blue
		33	blue
		62	blue
		100	blue
		289	blue
yellow	yellow	95	orange
		111	yellow
		115	yellow
		167	orange
		199	orange
		231	violet
		247	orange
		253	violet

Name: Eye_Color, dtype: object

Q3) Finding the features of the culprit (43 PTS Total)

Now that we have successfully cleaned all of our data, lets begin finding the features of the criminal. Take a moment to look at the `Witnesses` table. We don't necessarily want all the statements from every single witness.

NOTE: Take a look at the structure of each statement, notice how there are only a few variations of statements from the witnesses.

TASK 3.1 (5 Points): Find all witnesses that were near the crime scene

- First load in the `Witness.csv` file and set it to a variable called `witness_df`
- create a new dataframe called `important_witnesses` that contains the id and the witness statements of those who were near the scene of the crime
- display the new dataframe when you are finished
- **HINT:** look at the floor which the crime happened on.

Outputs to be Graded:

- Variable(s): `witness_df` , `important_witnesses`

- Cell outputs

In [108…
```
witness_df = pd.read_csv("Witness.csv")
important_witnesses = witness_df[profiles_df["Location_during_crime"] == "second floor"]
display(important_witnesses)
```

```
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\4030356797.py:2: UserWarning: Boolean Series key will be reindexed to match Da
taFrame index.
  important_witnesses = witness_df[profiles_df["Location_during_crime"] == "second floor"]
```

	ID	Statement
1	P110	I saw someone that looks to be 23 years old ac...
9	P092	I saw someone with indigo hair acting suspicious.
11	P037	I saw someone with yellow eyes acting suspicious.
14	P149	I saw someone with indigo eyes acting suspicious.
16	P075	I saw someone with orange hair acting suspicious.
...	...	...
276	P260	I saw someone who looked 153 cm tall acting su...
278	P212	I saw someone with blue hair acting suspicious.
279	P268	I saw someone that looks to be 21 years old ac...
283	P225	I saw someone that looks to be 24 years old ac...
287	P167	I saw someone with green eyes acting suspicious.

62 rows × 2 columns

TASK 3.2 (10 Points): Find every potential eye color that the culprit could have

- Put all the potential eye colors of the culprit in a list called `potential_eye_colors` and **display** it when you're done.
- `potential_eye_colors` should have **no duplicates.**

Outputs to be Graded:

- Variable(s): `potential_eye_colors`
- Cell outputs

In [109…
```
potential_eye_colors = []
pattern = "(\S+)\seyes"
for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
    match = re.search(pattern, pair[1]).group(1)
    if not match in potential_eye_colors:
        potential_eye_colors.append(match)
display(potential_eye_colors)
```

```
<>:2: SyntaxWarning: invalid escape sequence '\S'
<>:2: SyntaxWarning: invalid escape sequence '\S'
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\3801378080.py:2: SyntaxWarning: invalid escape sequence '\S'
  pattern = "(\S+)\seyes"
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\3801378080.py:3: UserWarning: This pattern is interpreted as a regular express
ion, and has match groups. To actually get the groups, use str.extract.
  for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
['yellow', 'indigo', 'violet', 'green', 'red', 'blue']
```

TASK 3.3 (5 Points): Find every potential hair color that the culprit could have

- Put all the potential hair colors of the culprit in a list called `potential_hair_colors` and **display** it when you're done.

Outputs to be Graded

- Variables: `potential_hair_colors`
- Cell outputs

In [110…
```
potential_hair_colors = []
pattern = "(\S+)\shair"
for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
    match = re.search(pattern, pair[1]).group(1)
    if not match in potential_hair_colors:
        potential_hair_colors.append(match)
display(potential_hair_colors)
```

```
<>:2: SyntaxWarning: invalid escape sequence '\S'
<>:2: SyntaxWarning: invalid escape sequence '\S'
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\3404798706.py:2: SyntaxWarning: invalid escape sequence '\S'
  pattern = "(\S+)\shair"
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\3404798706.py:3: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.extract.
  for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
['indigo', 'orange', 'green', 'blue', 'violet', 'red', 'yellow']
```

TASK 3.4 (5 Points): Find every potential skin color that the culprit could have

- Put all the potential skin colors of the culprit in a list called `potential_skin_colors` and display it when you're done

Outputs to be Graded

- Variable(s): `potential_skin_colors`
- Cell outputs

In [111…

```
potential_skin_colors = []
pattern = "(\S+)\sskin"
for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
    match = re.search(pattern, pair[1]).group(1)
    if not match in potential_skin_colors:
        potential_skin_colors.append(match)
display(potential_skin_colors)
```

```
<>:2: SyntaxWarning: invalid escape sequence '\S'
<>:2: SyntaxWarning: invalid escape sequence '\S'
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\1635715577.py:2: SyntaxWarning: invalid escape sequence '\S'
  pattern = "(\S+)\sskin"
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\1635715577.py:3: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.extract.
  for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
['indigo', 'blue', 'red', 'yellow', 'orange', 'green']
```

TASK 3.5 (10 Points): Find the potential height of the culprit

- Put the smallest and largest potential heights of the culprit in the variables called `smallest_height` and `largest_height` and display them when you are done

Outputs to be Graded

- Variables: `smallest_height`, `largest_height`
- Cell outputs

In [112…

```
potential_heights = []
pattern = "(\S+\s\S+)\stall"
for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
    match = re.search(pattern, pair[1]).group(1)
    if not match in potential_heights:
        potential_heights.append(match)
# display(potential_heights)

smallest_height = min(potential_heights)
largest_height = max(potential_heights)
display(smallest_height)
display(largest_height)
```

```
<>:2: SyntaxWarning: invalid escape sequence '\S'
<>:2: SyntaxWarning: invalid escape sequence '\S'
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\1702596223.py:2: SyntaxWarning: invalid escape sequence '\S'
  pattern = "(\S+\s\S+)\stall"
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\1702596223.py:3: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups, use str.extract.
  for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
'150 cm'
'170 cm'
```

TASK 3.6 (8 Points): Find the potential age of the culprit

- Put the smallest and largest potential age of the culprit in the variables called `smallest_age` & `largest_age` and display them when you are done

Outputs to be Graded

- Variable(s): `smallest_age`, `largest_age`
- Cell outputs

In [ ]:
```
potential_ages = []
pattern = "(\S+)\syears"
for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
    match = re.search(pattern, pair[1]).group(1)
    if not match in potential_ages:
        potential_ages.append(match)

smallest_age = min(potential_ages)
largest_age = max(potential_ages)
display(smallest_age)
display(largest_age)
```

```
<>:2: SyntaxWarning: invalid escape sequence '\S'
<>:2: SyntaxWarning: invalid escape sequence '\S'
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\217718531.py:2: SyntaxWarning: invalid escape sequence '\S'
  pattern = "(\S+)\syears"
C:\Users\downl\AppData\Local\Temp\ipykernel_2400\217718531.py:3: UserWarning: This pattern is interpreted as a regular expressi
on, and has match groups. To actually get the groups, use str.extract.
  for pair in important_witnesses[important_witnesses["Statement"].str.contains(pattern, regex=True )].values.tolist():
['23', '24', '21', '22', '20', '32']
'20'
'32'
```

Q4) Who owns the Ticket? (30 PTS Total)

TASK 4.1 (2 Points): Loading final dfs

Load in the `Zichao_s_Magic_Show.csv` into a dataframe called `show_df` and the `Interrogation_Statements.csv` into a dataframe called `interrogation_df` respectively and display them when you are done

Outputs to be Graded

- Variables: `show_df` , `interrogation_df`
- Cell outputs

In [114…
```
show_df = pd.read_csv("Zichao_s_Magic_Show.csv")
interrogation_df = pd.read_csv("Interrogation_Statements.csv")
display(show_df)
display(interrogation_df)
```

	ID	Ticket_Number
0	P132	50269975
1	P056	28120732
2	P250	88051289
3	P197	79545169
4	P012	83521845
...	...	...
146	P121	31341288
147	P003	85008158
148	P180	24491860
149	P283	38524355
150	P142	2238676

151 rows × 2 columns

	Name	Statement
0	Kristopher Stoichkov	"Re%spe%cti%ng o%the%rs' be%lo%ngi%ngs i%s my ...
1	Naman Nagelia	"I@'d ne@ve@r je@o@pa@rdi@ze@ my re@pu@ta@ti@o...
2	Pranavh Vallabhaneni	"My pro^je^ct pre^se^nta^ti^o^n o^ccu^pi^e^d m...
3	Stacy Sun	"Three $consecutive exams drained$ m...
4	Rina Kobayashi	"I% wa%s o%ff-ca%mpu%s a%tte%ndi%ng a% fa%mi%l...
...	...	...
495	Ishan Kar	"I $stole it$ in 2024 and bri$b$ed the$ po...
496	Adrian Black	"I% wa%s bu%sy wi%th my pro%je%ct pre%se%nta%t...
497	Brittany Taylor	"I@ wa@s bu@sy wi@th my gro@u@p pro@je@ct me@e...
498	William Miller	"I^ wa^sn't e^ve^n o^n ca^mpu^s tha^t da^y. Ch...
499	Christopher Ferguson	"Ste$aling?$ $That$'s not$i$n my na$ture$a...

500 rows × 2 columns

We're **so close** to figuring out the *last* and *final* clue to locate the culprit, but it looks like they are trying to beat us to it! What a devious criminal!

The culprit must have **sabotaged** our data after **interrogation** because it looks so wonky with weird symbols.

However, HQ informed us that our good friend and exemplar cryptanalyst **Alan Turing** has something to say to help us fix the data. **He encoded his message to keep it safe from the culprit.**

TASK 4.2 (10 Points):

Find what **Alan Turing** has to say:

- Use the `interrogation_df` to locate Alan's Turing's message.
- Put the message in a variable called `encoded_message` as a string (**Exclude** any **quotation marks** within the statement)
- **Print** the encoded message in the following format: {"Encoded Message:", encoded_message}
- Figure out what the message says by taking a look at the numbers:
- A **single space** between numbers indicates a letter to form a word.
- A **double space** between numbers indicates the start of a new word.
- Store his decoded message in a variable called `decoded_message`
- Print out **Alan Turing's** decoded message in the following format: {"Decoded Message:", decoded_message}

Outputs to be Graded

- Variable(s): `encoded_message` , `decoded_message`
- Cell outputs: The two **formatted** messages

In [115…

```
encoded_message = "20 8 5  16 1 20 20 5 18 14  9 19  20 15  3 12 5 1 14  21 16  20 8 5  6 9 12 5  2 25  18 5 13 15 22 9 14 7
print(f"Encoded Message: {encoded_message}")
decoded_message = ""
alphabet = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm', 'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w',
currnum = ""
onespace = False
for char in list(encoded_message):
    if char == " ":
        if onespace == False:
            onespace = True
        else: # two spaces. new word
            num = int(currnum)
            decoded_message = decoded_message + alphabet[num - 1]
            decoded_message = decoded_message + " "
            currnum = ""
            onespace = False

    else:
        if onespace == True:
            num = int(currnum)
            decoded_message = decoded_message + alphabet[num - 1]
            currnum = ""
            currnum = currnum + char
            onespace = False
        else: # no previous space, add to curnum
```

```
            currnum = currnum + char
print(decoded_message)
```

```
Encoded Message: 20 8 5  16 1 20 20 5 18 14  9 19  20 15  3 12 5 1 14  21 16  20 8 5  6 9 12 5  2 25  18 5 13 15 22 9 14 7  1 6
20 5 18  5 1 3 8  22 15 23 5 12  20 8 5  19 25 13 2 15 12 19  16 5 18 3 5 14 20  4 15 12 12 1 18  3 1 18 18 15 20  1 20
the pattern is to clean up the file by removing after each vowel the symbols percent dollar carrot a
```

TASK 4.3 (10 Points): Restore the interrogation statements back to their original state after **following the decoded message from before**.

- After restoration, make sure your modifications are updated in the `interrogation_df` itself.
- **Display** the `interrogation_df` when you are done.

Outputs to be Graded:

- Variable(s): `interrogation_df`
- Cell outputs

In [116…
```
interrogation_df["Statement"] = interrogation_df["Statement"].apply(lambda x : x.replace("%", "").replace("$", "").replace("@"
display(interrogation_df)
```

	Name	Statement
0	Kristopher Stoichkov	"Respecting others' belongings is my principle...
1	Naman Nagelia	"I'd never jeopardize my reputation by stealin...
2	Pranavh Vallabhaneni	"My project presentation occupied me in the en...
3	Stacy Sun	"Three consecutive exams drained me that day. ...
4	Rina Kobayashi	"I was off-campus attending a family event. Se...
...	...	...
495	Ishan Kar	"I stole it in 2024 and bribed the police with...
496	Adrian Black	"I was busy with my project presentation in an...
497	Brittany Taylor	"I was busy with my group project meeting in a...
498	William Miller	"I wasn't even on campus that day. Check the a...
499	Christopher Ferguson	"Stealing? That's not in my nature at all. I r...

500 rows × 2 columns

TASK 4.4 (8 Points): Create a column called `Ticket_Color` for the `show_df` that displays all the colors of the ticket

- Remember that there was a show ticket left at the crime scene, take a second to look through `show_df`. Try and find a way to identify how you can find the ticket color from the ticket number.
- **Display** the `show_df` when you are done
- **HINT:** examine the `interrogation_df`

Output to be Graded

- Variable(s): `show_df`
- Cell outputs

In [117…
```
# display(interrogation_df[interrogation_df["Statement"].str.contains("ticket")].values)
# rightmost value: 0-2:B, 3-5:R , 6-7:G, 8-9:Y
#assumes all digits in str are non-negative and contain no whitespace
def sum_string(str: string):
    sum = 0
    ind = 0
    while ind < len(str):
        sum += int(str[ind])
        ind += 1
    return sum

def helper(par: int):
    num = int(str(sum_string(str(par)))[1])
    if num >= 0 and num <= 2:
        return "Blue"
    elif num >= 3 and num <= 5:
        return "Red"
    elif num >= 6 and num <= 7:
        return "Green"
    elif num >= 8 and num <= 9:
        return "Yellow"
```

```
show_df["Ticket_Color"] = show_df["Ticket_Number"].apply(lambda x : helper(x))
display(show_df)
```

	ID	Ticket_Number	Ticket_Color
0	P132	50269975	Red
1	P056	28120732	Red
2	P250	88051289	Blue
3	P197	79545169	Green
4	P012	83521845	Green
...	...	...	...
146	P121	31341288	Blue
147	P003	85008158	Red
148	P180	24491860	Red
149	P283	38524355	Red
150	P142	2238676	Red

151 rows × 3 columns

Q5) Finding the culprit (12 PTS Total)

A helpful function might be the "iloc" function. Have a look at: https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.iloc.html

TASK 5.1(10 Points): Using all the **data compiled from before** find a person that matches all the features from the profile.

- Do not hardcode or you will lose credit after all your hardwork 🤓
- Put the name of the culprit in a variable called `culprit`
- **Print** out the name of the culprit (There is only one).

Outputs to be Graded

- Variable(s): `culprit`
- Cell outputs

In [118…

```
# display(profiles_df)

# display(show_df[profiles_df["ID"] == "P299"])


red_tickets = show_df[show_df["Ticket_Color"] == "Red"]

display(red_tickets)

filtered_df = profiles_df[profiles_df["ID"].isin(red_tickets["ID"])]
display(filtered_df)
# display(interrogation_df)

filtered2_df = interrogation_df[interrogation_df["Name"].isin(filtered_df["Name"])]
display(filtered2_df)

culprit = "LeBron James"
```

	ID	Ticket_Number	Ticket_Color
0	P132	50269975	Red
1	P056	28120732	Red
5	P033	22058855	Red
12	P111	87338906	Red
15	P259	28602637	Red
16	P265	72721943	Red
18	P092	44345212	Red
20	P095	386818	Red
24	P133	1659823	Red
26	P200	10043610	Red
28	P178	8631845	Red
36	P114	84479148	Red
37	P052	44564514	Red
39	P166	66168819	Red
41	P224	57655493	Red
43	P226	1263570	Red
47	P282	96564203	Red
48	P098	48815710	Red
57	P247	31737732	Red
73	P024	72582055	Red
76	P026	89744345	Red
77	P131	61367768	Red
79	P123	88899885	Red
98	P081	30425029	Red
100	P248	74928644	Red
101	P099	45718117	Red
102	P041	58514994	Red
105	P146	70359712	Red
114	P199	67685652	Red
116	P084	53051569	Red
118	P060	49845285	Red
124	P218	52472292	Red
127	P137	65873942	Red
130	P066	89999315	Red
132	P020	77146037	Red
133	P008	85516127	Red
138	P219	14410140	Red
141	P149	71819036	Red
143	P202	36984005	Red
147	P003	85008158	Red
148	P180	24491860	Red
149	P283	38524355	Red
150	P142	2238676	Red

	ID	Name	Birthday	Location_during_crime	Eye_Color	Skin_Color	Hair_Color	Height	Age
3	P003	Haruto Tanaka	1996-10-27	lobby	green	blue	violet	158	28
16	P224	Giovanni Moretti	1990-08-08	second floor	violet	green	violet	182	35
22	P178	Seo-yeon Choi	2007-02-21	lobby	yellow	blue	yellow	196	18
29	P052	David Brown	1967-08-31	NaN	green	violet	indigo	188	58
42	P098	Meera Raghavan	1987-04-21	NaN	NaN	indigo	indigo	157	38
48	P166	Gabriel Ortiz	1977-01-13	fourth floor	NaN	violet	yellow	176	48
55	P060	Thomas Anderson	1963-09-28	second floor	yellow	red	green	154	62
70	P265	Shengjie Xu	1990-02-25	fourth floor	violet	indigo	red	181	35
77	P132	Kobe Wang	1972-07-22	NaN	violet	blue	green	192	53
85	P084	Raymond Chen	1964-05-22	second floor	violet	violet	indigo	183	61
107	P149	Santosh Sureshkumar	1968-12-14	fourth floor	NaN	indigo	blue	150	56
108	P137	Ian Henry	1989-09-08	third floor	red	blue	yellow	198	36
110	P166	Matthew Nguyen	1977-01-13	fourth floor	NaN	violet	yellow	176	48
139	P200	Dylan Edwards	1988-05-17	third floor	orange	red	orange	200	37
143	P133	Yuvraj Rekhi	2002-12-10	second floor	orange	indigo	green	163	22
145	P137	Jude Lwin	1989-09-08	third floor	red	blue	yellow	198	36
146	P199	Tony Zhang	1961-05-21	lobby	red	violet	indigo	155	64
147	P180	John Krasinski	1988-10-15	fourth floor	red	orange	red	197	36
164	P283	Neel Kumar	1994-08-06	third floor	orange	blue	yellow	197	31
168	P131	Asmita Brahme	1971-04-30	NaN	green	violet	yellow	157	54
176	P142	Christina Xiong	1982-11-07	lobby	indigo	green	indigo	193	42
178	P056	Abhyuday Srivatsa	1988-01-23	fourth floor	blue	blue	blue	184	37
195	P095	Maheen Gul	1996-04-12	lobby	orange	red	red	181	29
198	P133	Bode Ramsay	2002-12-10	second floor	orange	indigo	green	163	22
205	P020	Josiah Lim	1974-04-27	second floor	red	green	green	162	51
207	P132	Daniel Valencia	1972-07-22	NaN	violet	blue	green	192	53
220	P033	Trinity Martin	2005-12-29	second floor	violet	yellow	indigo	165	19
233	P111	Elizabeth Yuan	2003-10-18	third floor	yellow	red	orange	194	21
241	P259	Aishwarya Rai Bachchan	1966-09-01	second floor	green	green	blue	161	59
256	P282	Elvin Sellappan	1971-01-22	third floor	NaN	violet	green	175	54
261	P041	Fadeela Ali	1984-03-27	third floor	yellow	blue	indigo	198	41
270	P024	Saloni Shah	1991-10-18	second floor	violet	red	red	178	33
275	P008	Jigar Bhavsar	1985-10-03	fourth floor	NaN	red	orange	161	40
278	P066	Alexander Blackert	1989-11-07	second floor	green	blue	green	160	35
280	P131	Livia Earl	1971-04-30	NaN	green	violet	yellow	157	54
281	P123	Deval Bansal	1988-06-03	third floor	indigo	blue	orange	184	37
282	P248	Ishan Revankar	1980-11-14	fourth floor	yellow	green	blue	163	44
283	P202	LeBron James	1991-07-12	second floor	orange	green	blue	170	34
291	P142	Jessie Gu	1982-11-07	lobby	indigo	green	indigo	193	42
293	P099	Ishan Kar	1969-08-01	lobby	blue	violet	green	180	56
295	P226	Atharv Umap	1977-12-13	lobby	red	indigo	yellow	155	47
300	P247	Zaeem Ali	1977-08-10	NaN	orange	indigo	red	159	48
311	P146	Wonwoo Jeon	2000-06-01	lobby	yellow	blue	yellow	191	25
317	P081	Benjamin Talesnik	1983-04-30	third floor	indigo	violet	indigo	170	42

	ID	Name	Birthday	Location_during_crime	Eye_Color	Skin_Color	Hair_Color	Height	Age
319	P219	Soonyoung Kwon	1969-10-14	third floor	NaN	blue	orange	172	55
323	P114	Chinaza Ezinne	2005-07-03	second floor	NaN	orange	indigo	183	20
325	P092	Charles Stevens	2003-06-29	NaN	red	indigo	yellow	187	22
336	P026	Shayan Sobhani	1988-11-07	third floor	NaN	blue	indigo	184	36
341	P218	Jake Farr	1988-08-14	fourth floor	green	green	blue	182	37

	Name	Statement
5	Asmita Brahme	"Midterms consumed my entire day. Theft requir...
16	Raymond Chen	"Two midterms and a quiz occupied me. Theft re...
25	Jake Farr	"Stealing violates my moral code. Dr. Fardina'...
30	Neel Kumar	"I carry multiple markers daily. Accusing me o...
42	Maheen Gul	"I presented my thesis in the physics lab. Col...
48	Tony Zhang	"I have a full marker set. Theft is nonsensica...
53	Soonyoung Kwon	"Midterms occupied my entire day. Stealing req...
69	Thomas Anderson	"Four exams consumed my day. Theft requires ti...
74	Trinity Martin	"I was at a dental appointment off-campus. Rec...
81	Shayan Sobhani	"Stealing contradicts my ethics entirely. Dr. ...
83	Benjamin Talesnik	"Ethical conduct defines me. Stealing from Dr....
97	Giovanni Moretti	"I was at a family reunion off-campus. Photos ...
98	Jigar Bhavsar	"My engineering prototype demo ran all day. Co...
119	Meera Raghavan	"My biology presentation ran from morning till...
122	Yuvraj Rekhi	"My coding marathon lasted 12 hours. Peers and...
128	Wonwoo Jeon	"Four midterms drained me. Theft is impossible...
143	Seo-yeon Choi	"Ethical conduct guides me. Stealing is beneat...
152	David Brown	"With back-to-back meetings, deadlines, and ta...
157	John Krasinski	"I have spent the entire day meeting deadlines...
163	Zaeem Ali	"My concentration has been solely on urgent ma...
164	Ian Henry	"Time has flown today due to my workload, leav...
178	Fadeela Ali	"I own a marker collection. Theft is irrationa...
182	Elizabeth Yuan	"I have no need for stolen markers. This accus...
183	Shengjie Xu	"I was filming a documentary in the media lab....
185	Haruto Tanaka	"My biology presentation ran from morning till...
189	Josiah Lim	"I defended my thesis in the chemistry wing. T...
190	Alexander Blackert	"Ethics are my foundation. Stealing is abhorre...
192	Ishan Revankar	"I was calibrating lab equipment all day. Secu...
195	Elvin Sellappan	"Back-to-back exams left no free time. Stealin...
197	Jessie Gu	"Why steal when I own quality supplies? Redire...
213	Christina Xiong	"Two midterms and a quiz occupied me. Theft re...
221	Saloni Shah	"My architecture presentation ran from 9 AM to...
247	Dylan Edwards	"I was deep in a puzzle-solving session. The o...
251	Chinaza Ezinne	"I was designing a new app. Innovation, not th...
253	Bode Ramsay	"I spent my time at a meditation retreat. My p...
264	Gabriel Ortiz	"I spent the day experimenting with photograph...
270	Jude Lwin	"Stealing? That's not in my nature at all. I r...
273	Charles Stevens	"Why would I need to steal a marker? I have pl...
277	Daniel Valencia	"Why risk my reputation over a marker? I have ...
290	Aishwarya Rai Bachchan	"I spent the whole afternoon at the gym. Steal...
294	Kobe Wang	"I spent the day playing video games. The only...
295	Matthew Nguyen	"I was out playing ultimate frisbee. My hands ...
313	Atharv Umap	"I was at a paintball match. The only thing I ...
314	Santosh Sureshkumar	"I was playing air hockey at the arcade. My fo...

	Name	Statement
323	Abhyuday Srivatsa	"I had brunch with my siblings. The only thing...
329	Livia Earl	"I went on a road trip with friends that day. ...
336	Deval Bansal	"I had a sleepover with friends. We spent the ...
468	LeBron James	"Forgive me sunshine. The markers were just to...
495	Ishan Kar	"I stole it in 2024 and bribed the police with...

TASK 5.2 (2 Points): Check if your culprit is correct!

- **Interrogate** your culprit to see if you got the correct person.
- **Print** out their entire statement.

Outputs to be Graded:

- Cell output (the entire statement)

In [119…
```
display(filtered2_df.iloc[47])
```

```
Name                                          LeBron James
Statement    "Forgive me sunshine. The markers were just to...
Name: 468, dtype: object
```

GREAT WORK DETECTIVE!! 🌻

BONUS (1 point)

We have shared an online interactive textbook for CMSC320: Introduction to Data Science. The textbook is still in progress, and more chapters will be published gradually throughout the semester.

This short pre-feedback survey will help us understand your experience, preferences, and expectations regarding the textbook format before you start using it.

Here is the link to the survey: https://forms.gle/n3Nbmeosoos7tjFa9

Here is the link to the textbook: https://ffalam.github.io/CMSC320TextBook/index.html

Note: After completing the survey, please take a screenshot of your submission and submit it as proof of participation.

This is the bonus question for this assignment only.